

Chaal

Technical Report AMD DevMaster Hackathon, Track 3: Physical AI Challenge

Application	Chaal
Team	himanshu748 (solo)
Embodiment	Unitree Go2 quadruped, 12 actuated joints
Task	Velocity-command tracking (legged locomotion)
Hardware	AMD Radeon PRO, gfx1100 (RDNA 3, Navi 31), 48 GB VRAM, 48 CUs
Stack	ROCm 7.2.4, PyTorch 2.10.0+rocm7.2.4, Genesis 1.3.0 (gs . amdgpu), rsl_rl 5.4.2
Licence	Apache-2.0

1. Target application

A quadruped robot is given a velocity command in its own body frame, meaning a forward speed, a sideways speed and a turn rate, and has to produce a gait that follows it without falling over. This is the control problem underneath every practical use of a legged robot: inspection rounds in a plant, carrying a payload over ground a wheeled robot cannot cross, or reaching somewhere a person should not have to go.

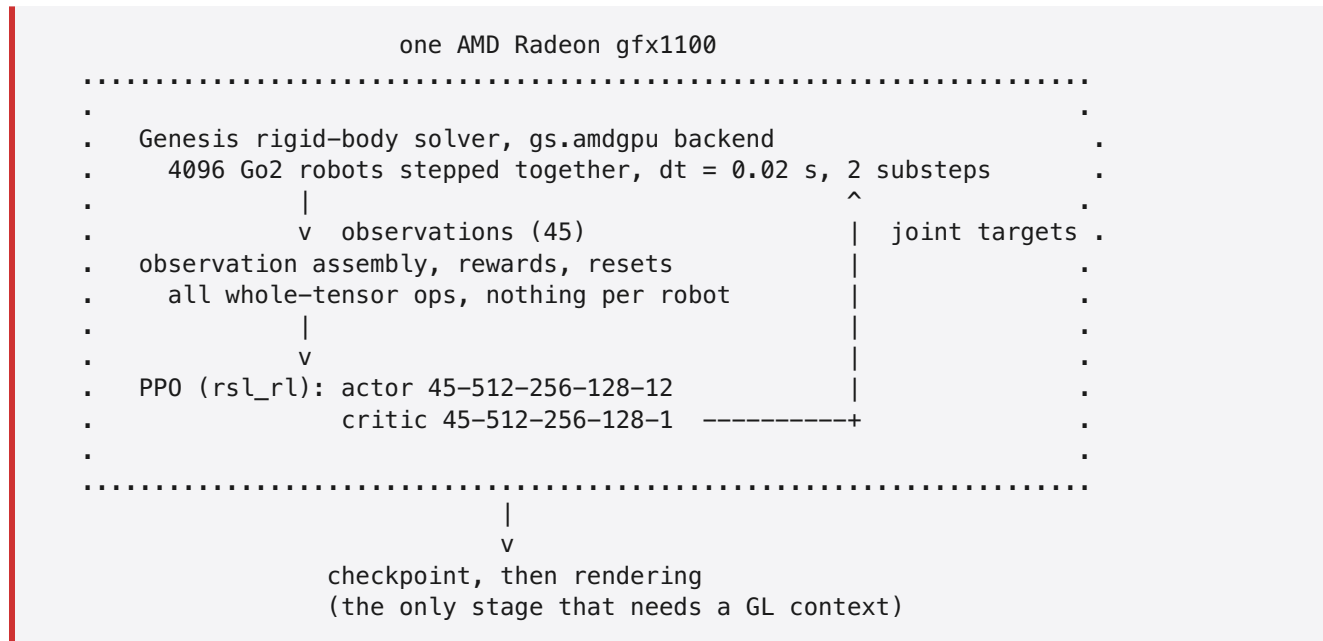
It is chosen here for three reasons.

It cannot be faked. A manipulation demo can be staged with scripted waypoints. A quadruped either walks or it lies on the floor, and the failure is visible in one frame of video.

It has a metric that resists gaming. The obvious measure, how long the robot stays upright, is satisfied perfectly by a policy that stands still and never moves. So every result in this report reports velocity tracking error beside survival, and a policy that freezes scores badly on the first even while scoring perfectly on the second.

It needs the GPU for the reason the GPU is interesting. Locomotion is learned from scratch, from tens of millions of environment steps. That is not a workload you can do on a laptop, and it makes the Radeon load-bearing rather than decorative.

2. System architecture



The important property is what is *absent*: there is no host round trip in the loop. Genesis writes state into GPU tensors, the observation and reward code is whole-tensor arithmetic over those same tensors, and `rsl_rl` reads them directly. The only thing that crosses back to the CPU during training is one log line per iteration.

Observation, 45 values. Base angular velocity (3), projected gravity (3), the command (3), joint positions relative to the standing pose (12), joint velocities (12), previous action (12). Base linear velocity is deliberately excluded, since it is not cleanly measurable on a real Go2.

Action, 12 values. An offset per joint, scaled by 0.25 and added to the standing pose, tracked by a PD controller at k_p 20, k_d 0.5.

Episode. 20 simulated seconds, so 1000 steps. Terminated early if the body rolls or pitches past 10 degrees. Commands are resampled every 4 seconds within an episode, so a policy cannot memorise one gait and coast.

Reward. The standard velocity-tracking formulation: two exponential tracking terms for linear and angular velocity, plus shaping penalties on vertical velocity, action rate, deviation from the standing pose and body height. The reward design is not claimed as a contribution; it is the well-established set for this task, and it is used unchanged so that the measurements are about the platform rather than about reward engineering.

2.1 A note on what runs where

Rendering is kept strictly out of the training path. Training is state-based and touches no pixels; video is produced afterwards from a saved checkpoint by `scripts/render.py`. This matters more than it sounds, for the reason in section 6.1.

3. Data

There is no dataset, and that is a property of the approach rather than an omission. No demonstrations are collected, no teleoperation is recorded, and no pretrained policy is fine-tuned. The policy is trained from random initialisation on experience the simulator generates: **49,152,000 environment steps** for the headline run, all produced on the Radeon during the 8 minutes 41 seconds of training.

The only external asset is the Unitree Go2 URDF, which ships inside the Genesis wheel, so the project has no download step at all. This turned out to matter: the Radeon Cloud instance sits in mainland China, where `github.com` refuses connections and `huggingface.co` times out, so any project depending on fetching assets at runtime would not have run there without a mirror.

Evaluation uses no dataset either. Each condition in section 5 replays the same commands under a different physical perturbation.

4. How the Radeon is used

stage	on the Radeon	evidence
Physics simulation	yes, <code>gs.amdgpu</code> backend	<code>bench-results/scaling.json</code>
Observation, reward, reset logic	yes, GPU tensors throughout	<code>chaal/env.py</code>
Policy inference during rollout	yes	same process, same device
PPO gradient updates	yes	<code>runs/go2-4096/summary.json</code>
Rendering for the demo video	yes, offscreen via EGL	<code>scripts/render.py</code>

Every stage of the pipeline runs on the single GPU. There is no second device and no CPU fallback anywhere in the training loop.

4.1 Headline training run

Environments	4096 in parallel
Iterations	500
Wall clock	520.9 s (8 m 41 s)
Experience	49,152,000 environment steps
Throughput	94,363 env-steps/s including PPO updates
Time per iteration	1.042 s
Final mean reward	811.94
Mean episode length	999.78 of 1000

The learning curve is monotonic: -11.0, 21.6, 190.2, 549.3, 636.8, 689.2, 734.6, 725.8, 762.2, 778.6, ending at 811.9.

As section 1 warned, a mean episode length of 999.78 out of 1000 is exactly what a robot that has learned to stand still would produce. The reward decomposition rules that out: the linear velocity tracking term contributes 45.85, which inverts to an average tracking error near 0.14 m/s against commands drawn uniformly from -1 to +1 m/s.

4.2 Scaling, and where the bottleneck actually is

`chaa1 bench` measures two things separately at each environment count: physics stepping alone, and a full PPO iteration including rollout, advantage estimation and gradient steps.

environments	physics only	full training	s/iteration	peak VRAM	ratio
256	41,518/s	10,287/s	0.60	0.18 GB	4.0x
512	81,591/s	23,764/s	0.52	0.21 GB	3.4x
1024	152,303/s	42,282/s	0.58	0.26 GB	3.6x
2048	270,530/s	67,977/s	0.72	0.36 GB	4.0x
4096	460,756/s	93,546/s	1.05	0.57 GB	4.9x
8192	849,217/s	121,949/s	1.61	0.98 GB	7.0x
16384	1,337,088/s	138,330/s	2.84	1.77 GB	9.7x
32768	1,700,322/s	146,272/s	5.38	3.42 GB	11.6x

The two columns diverge, and that is the result. Physics throughput rises close to linearly with environment count, from 41.5k to 1.70M, a factor of 41. Training throughput does not: it climbs steeply to about 8192 environments and then flattens, gaining **13.4% from 8192 to 16384** and **5.7% from 16384 to 32768** while the simulation work required to feed it doubles at each step. The ratio between the columns widens from 3.4x to 11.6x.

The practical reading is that **past roughly 8192 environments the simulator stops being the bottleneck and the learning update becomes it**, so additional parallelism buys simulation throughput that the learner cannot consume. This is worth knowing before choosing a configuration, and it is invisible if you benchmark only the simulator, which is the usual thing to publish.

The second reading is that **memory is not the constraint anyone should be planning around here**. The largest configuration, 32768 robots stepping in parallel, peaks at 3.42 GB on a 48 GB card. The limit is arithmetic in the update, not capacity, so a card with less VRAM and the same compute would reach the same ceiling.

Reporting only the physics column would have made this project look between 3 and 12 times better than it is, depending on where the number was taken. Both columns are measured, both are in `bench-results/scaling.json`, and the headline throughput quoted anywhere in this report is always the training one.

5. Generalisation and robustness

Training randomises the velocity command and nothing else. Mass, friction and external disturbances are never varied during training, so every condition below except the baseline is genuinely off-distribution. 512 environments, 10 simulated seconds per condition, one policy.

condition	survival	tracking error	return
baseline	99.8%	0.096 m/s	477.1
payload 3 kg	100.0%	0.106 m/s	470.0
payload 6 kg	99.8%	0.123 m/s	450.7
friction 0.5	100.0%	0.113 m/s	466.0
friction 0.3	97.9%	0.175 m/s	431.0
friction 0.2	91.2%	0.288 m/s	363.3
push 0.5 m/s	99.8%	0.117 m/s	462.4
push 1.0 m/s	98.6%	0.147 m/s	444.3

Degradation is graceful and monotonic in every axis. A 6 kg payload, on a robot of roughly 15 kg, costs 28% of tracking accuracy and essentially no falls. The policy holds up until friction drops to 0.2, where it starts to lose footing.

Metrics are masked by whether a robot is still standing, so a fallen robot stops accruing credit rather than contributing a flattering tracking error while lying still.

6. Contributions

6.1 A silent no-op that produced a false robustness result

The first version of the friction rows above showed a "low friction" condition scoring **identically to baseline**, 0.095 m/s tracking error against 0.095, and 478.0 return against 478.6. That is not what a slippery floor does.

The cause: `robot.set_friction(0.2)` does change the robot's own geometry from 1.0 to 0.2, and changes contact friction not at all, because **Genesis resolves a contact pair by taking the larger of the two surfaces' values** and the ground plane was still at its default of 1.0.

Confirmed by direct measurement rather than by reading code, since the rule is in compiled kernels:

robot mu	plane mu	survival	tracking error
1.0	1.0	1.000	0.0972 m/s
0.2	1.0	1.000	0.0973 m/s
1.0	0.2	1.000	0.0960 m/s
0.2	0.2	0.898	0.2977 m/s

Either surface alone reproduces the baseline to three decimals. Both together cost ten points of survival and triple the tracking error.

This is worth reporting because of how it fails. It does not raise, it does not warn, and it returns a plausible number. Any domain-randomisation sweep that randomises friction on the robot only has this bug and nothing in its output reveals it. Had it not been caught, this report would have contained a robustness claim that was the baseline measured a second time.

6.2 Genesis cannot build a scene without an OpenGL context

`scene.build()` constructs an offscreen renderer whether or not anything is rendered. `Visualizer.__init__` assigns its rasterizer context unconditionally, `Visualizer.build()` calls into it unconditionally, and neither `Scene` nor `VisOptions` exposes a flag to prevent it. `show_viewer=False` disables only the interactive viewer.

On a ROCm container, which has no reason to ship OpenGL libraries, this fails with `AttributeError: 'NoneType' object has no attribute 'eglQueryString'` from inside PyOpenGL, which never mentions the real cause. Reported upstream as [genesis-world#3129](#), scoped deliberately to the error message: a previous request to disable the visualizer outright was rejected by the maintainers, and this asks only that the failure name its cause and the packages that fix it.

This is why Chaal keeps rendering out of the training path entirely, and why `chaal doctor` checks for a GL platform before any GPU time is spent.

6.3 Upstream contributions

what	where
Unactionable GL error on ROCm containers, with reproduction and fix	genesis-world#3129 (new issue)
Measured evidence for the friction max-combination problem	genesis-world#2718 (comment)

The friction finding was independently derived here, but an existing open issue already identified the `max()` behaviour and proposed MuJoCo's `geom_priority` as the fix. Rather than file a duplicate on a 29,000-star repository, the measurements were contributed to that issue, which had no comments and argued the case qualitatively. It now carries an external empirical confirmation and the observation that the failure is silent.

6.4 The measurement discipline itself

Three of the numbers in this report exist because a plausible result was checked rather than accepted: the friction no-op, the difference between physics and training throughput, and the possibility that a near-perfect episode length meant a robot standing still. Each would have made the project look better if left alone.

7. Deliverables

deliverable	where
Technical report	<code>docs/report.md</code> , this document, and its PDF
Source code	<code>chaal/</code> , with a Dockerfile
Reproducibility README	<code>README.md</code>
Demo video	<code>demo-video.mp4</code>
Supplementary	<code>docs/upstream.md</code> , <code>bench-results/*.json</code>

Every number quoted here is reproducible with the three commands in the README, and the raw JSON behind each table is committed rather than transcribed.

8. Team

Solo entry by **himanshu748**. All design, implementation, measurement, and the upstream reports are my own work. Genesis, rsl_rl and the Go2 URDF are third-party open source, used as described above and credited in the README.